

PAP 2.2 – Python-Auswertung

Wichtige Funktionen für das PAP.

Alle in diesem Dokument geschriebenen Zeilen Code sind von mir eigenständig geschrieben. Für ein sauberes und strukturiertes Dokument wurde eine eigene Pythonlibrary geschrieben, welche insbesondere ans PAP angepasst wurde. Dieses ist unter dem Namen "Papulator" auf meinem GitHub zu finden. Diese soll stetig ausgearbeitet werden. Der Stand dieses Protokolls ist der 19.05.2026.

Inhaltsverzeichnis zur Auswertung

- [Definition der Versuchsvariablen](#)
 - [Import aller genutzen Libraries](#)
 - [Auswertung der Aufgaben](#)
 - [Aufgabe 1](#)
 - [Aufgabe 2](#)
 - [Aufgabe 3](#)
 - [Aufgabe 4](#)
 - [Aufgabe 5](#)
 - [Aufgabe 6](#)
-

Definition der Versuchsvariablen

Im Folgenden sind Versuchsvariablen definiert, die für einen besseren Workflow sorgen sollen. Diese werden zum exportieren und überschreiben von Dateien wichtig sein und ermöglichen es, diese Datei für jeden Versuch zu benutzen und lediglich die Variablen zu verändern. Es ist jedoch empfohlen eine Kopie der Forlage für jeden Versuch zu machen, damit dieses Dokument strukturiert bleibt.

Zudem sind wichtige Konstanten definiert, die immer wieder auftauchen.

```
In [1]: versuchsnummer = "243"
versuchsname = "White_Noise"
aufgabe = "0z"
plt_save_folder = f"../img/plots/"
```

Import aller genutzen Libraries

```
In [2]: # Eigene PythonLibrary für das Pap
import Papulator as pap
from Papulator import Colors as c
from Papulator import const
from Papulator import Sympy_Symbols as sym

# Numpy für bessere Berechnungen
import numpy as np
from numpy import exp, sqrt, log, pi
from uncertainties import unumpy as unp

# Weiteres für bessere Rechnungen
import pylab as py

import math

# Berechnungen und Plotting
from scipy import odr
import scipy.optimize as optimize
from scipy.optimize import curve_fit
from scipy.stats import norm
from scipy.stats import chi2
from scipy.stats import poisson
from scipy.signal import find_peaks
from scipy.signal import argrelextrema, argrelemin, argrelemax
from scipy.special import factorial
from scipy.integrate import quad

import matplotlib.pyplot as plt
import matplotlib.mlab as mlab
import matplotlib.transforms as transforms

import astropy.units as u

# Zum Auslesen von Dateien und ähnlichem
import os
import os.path

import pandas as pd
import csv
import re
```

```
from io import StringIO

# Besseres Funktionen handling
import sympy as sp
from sympy import separatevars
```

Auswertung der Aufgaben

Aufgabe 1

```
In [3]: aufgabe = '1a'
pap.plot_general_csv("A1/5KOhm_FFT.csv", "Time (s)", "Channel 1 (V)",
                    title="Frequenzgang", xlabel="Frequenz [Hz]", ylabel="dBV",
                    log_x=True, save_path=fr"{plt_save_folder}Frequenzgang_{aufgabe}.pdf", show_plot=False)

aufgabe = '30K'
pap.plot_general_csv("A1/Spectrum_30k.csv", "Frequency (Hz)", "Trace 1 (dBV)",
                    title="Frequenzgang", xlabel="Frequenz [Hz]", ylabel="dBV",
                    log_x=True, save_path=fr"{plt_save_folder}Frequenzgang_{aufgabe}.pdf")
```

c:\Folders\Uni-Heidelberg\Praktika\PAP2\IDE_PAP2\PAP-2\Versuche\243\Python\Papulator.py:737: UserWarning: No artists with labels found to put in legend.
Note that artists whose label start with an underscore are ignored when legend() is called with no argument.

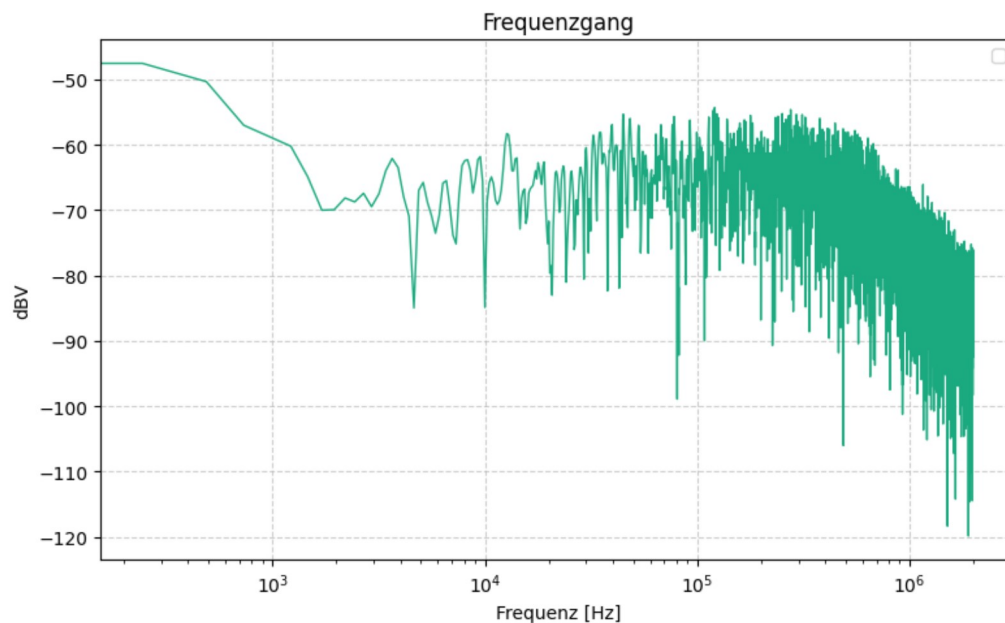
ax.legend()

Plot saved to ../img/plots/Frequenzgang_1a.pdf

c:\Folders\Uni-Heidelberg\Praktika\PAP2\IDE_PAP2\PAP-2\Versuche\243\Python\Papulator.py:737: UserWarning: No artists with labels found to put in legend.
Note that artists whose label start with an underscore are ignored when legend() is called with no argument.

ax.legend()

Plot saved to ../img/plots/Frequenzgang_30K.pdf

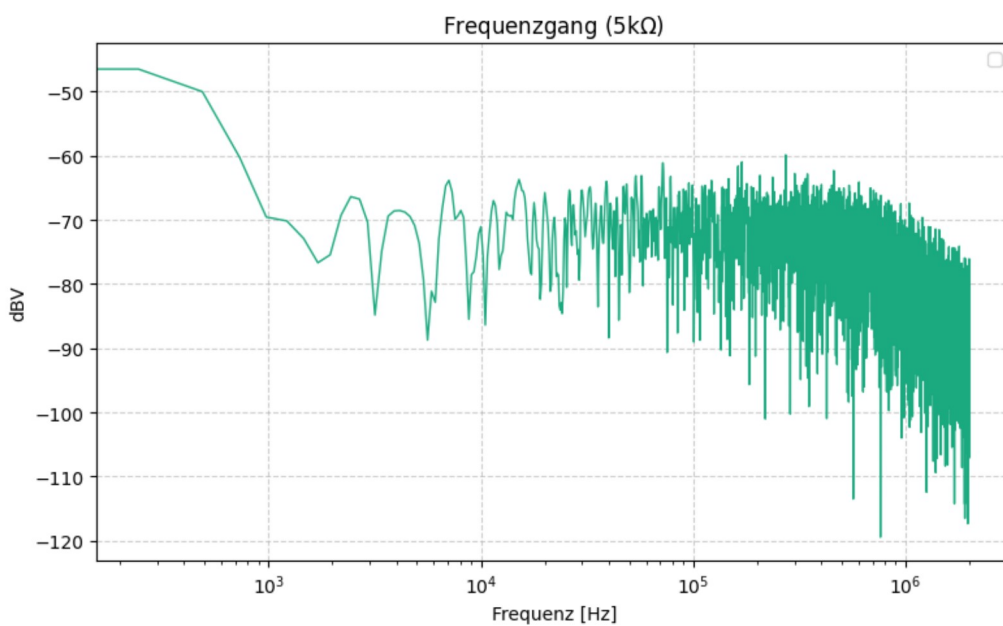


```
In [4]: aufgabe = '5K'
pap.plot_general_csv("A1/Spectrum_5k.csv", "Frequency (Hz)", "Trace 1 (dBV)",
                    title=fr"Frequenzgang (5k$\Omega$)", xlabel="Frequenz [Hz]", ylabel="dBV",
                    log_x=True, save_path=fr"{plt_save_folder}Frequenzgang_{aufgabe}.pdf")
```

c:\Folders\Uni-Heidelberg\Praktika\PAP2\IDE_PAP2\PAP-2\Versuche\243\Python\Papulator.py:737: UserWarning: No artists with labels found to put in legend.
Note that artists whose label start with an underscore are ignored when legend() is called with no argument.

ax.legend()

Plot saved to ../img/plots/Frequenzgang_5K.pdf

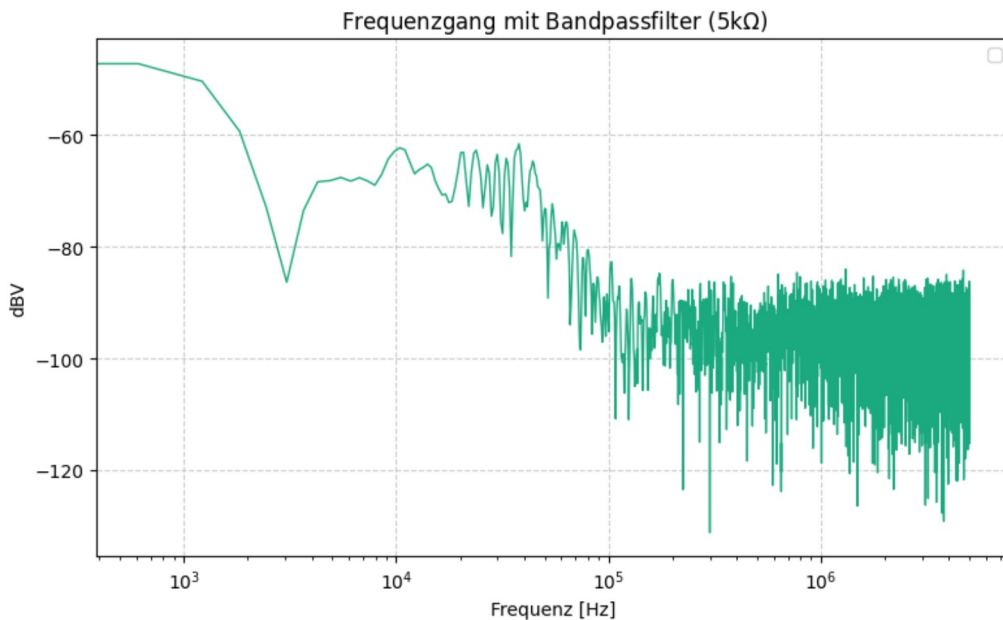


```
In [5]: aufgabe = 'Bandfliter-5K'
pap.plot_general_csv("A1/Spectrum_5k_MitDASBandpassfilter.csv", "Frequency (Hz)", "Trace 1 (dBV)",
                    title=fr"Frequenzgang mit Bandpassfilter (5k$\Omega$)", xlabel="Frequenz [Hz]", ylabel="dBV",
                    log_x=True, save_path=fr"{plt_save_folder}Frequenzgang_{aufgabe}.pdf")
```

c:\Folders\Uni-Heidelberg\Praktika\PAP2\IDE_PAP2\PAP-2\Versuche\243\Python\Papulator.py:737: UserWarning: No artists with labels found to put in legend.
Note that artists whose label start with an underscore are ignored when legend() is called with no argument.

ax.legend()

Plot saved to ../img/plots/Frequenzgang_Bandfliter-5K.pdf



Aufgabe 2

```
In [6]: aufgabe = '2a'
# Eintragen der Messdaten
n_messungen = 127

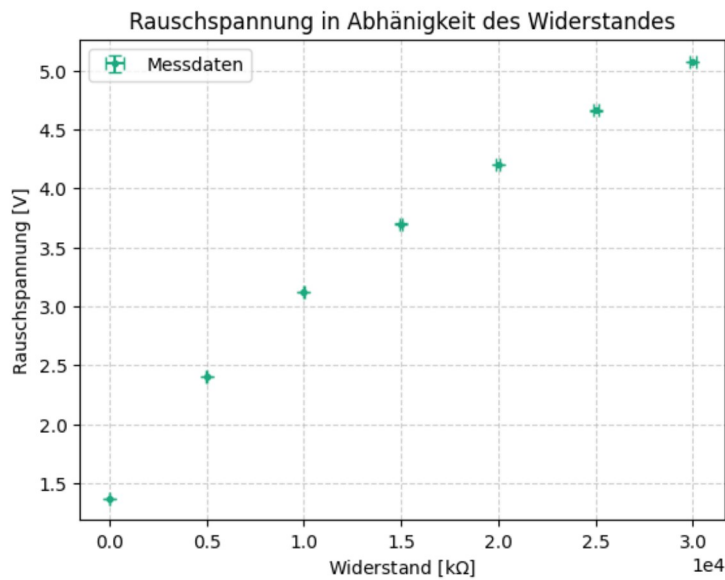
# Widerstände vo kOhm in Ohm
wid_stands = np.array([0, 5, 10, 15, 20, 25, 30]) * 1e3

# Spannung in V (oder in dBV?)
avg_Vr = np.array([1.3702, 2.4042, 3.1159, 3.6989, 4.201, 4.6594, 5.0751])
sig_Vr = np.array([0.0054, 0.0093, 0.011, 0.0149, 0.0147, 0.0173, 0.0188])
err_Vr = sig_Vr/np.sqrt(n_messungen)

# Fehler der Bauteile:
err_wid = 0.005 # pm 0.5% Fehler für Widerstände
err_vm = 0.003 # pm 0.3% Genauigkeit des Voltmeters

plt.errorbar(
    wid_stands,
    avg_Vr,
    err_Vr,
    wid_stands * err_wid,
    label=fr'Messdaten',
    fmt='.',
    ls='-',
    capsize=3.5
)
```

```
pap.plot_me('Rauschspannung in Abhängigkeit des Widerstandes', r'Widerstand [ $\mathrm{k}\Omega$ ]', r'Rauschspannung [V]', fr'{plt_save_folder}Spann_va_Wid.
```

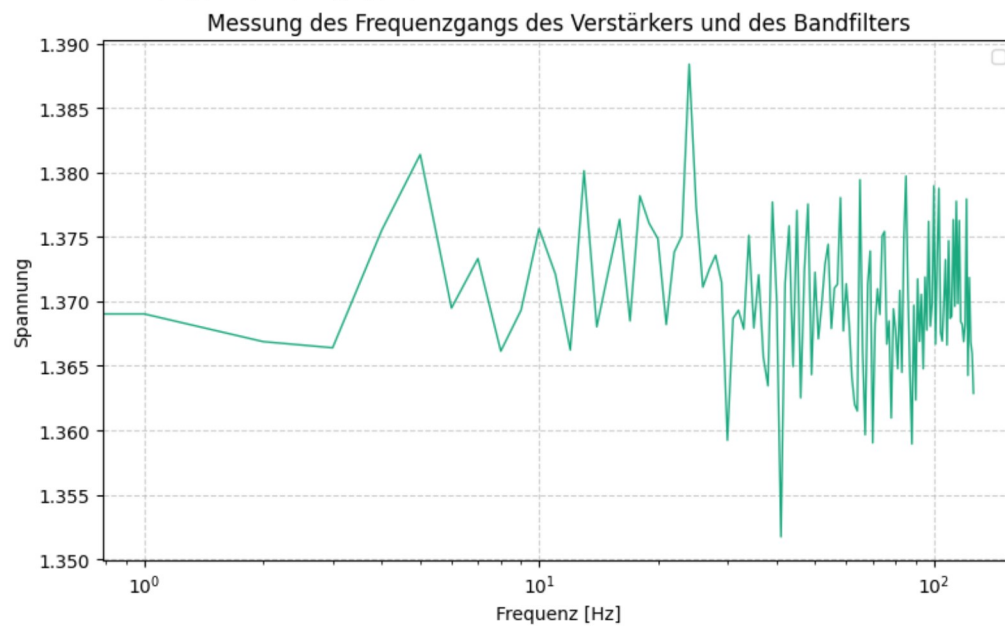


```
Out[6]: <Axes: title={'center': 'Rauschspannung in Abhängigkeit des Widerstandes'}, xlabel='Widerstand [ $\mathrm{k}\Omega$ ]', ylabel='Rauschspannung [V]'
```

```
In [7]: aufgabe = '2a'
pap.plot_general_csv("A2/0k.csv", "x000", "y000",
                    title=fr"Messung des Frequenzgangs des Verstärkers und des Bandfilters", xlabel="Frequenz [Hz]", ylabel="Spannung",
                    log_x=True, save_path=fr'{plt_save_folder}Frequenzgang_{aufgabe}.pdf')
```

c:\Folders\Uni-Heidelberg\Praktika\PAP2\IDE_PAP2\PAP-2\Versuche\243\Python\Papulator.py:737: UserWarning: No artists with labels found to put in legend.
Note that artists whose label start with an underscore are ignored when legend() is called with no argument.

```
ax.legend()
Plot saved to ../img/plots/Frequenzgang_2a.pdf
```

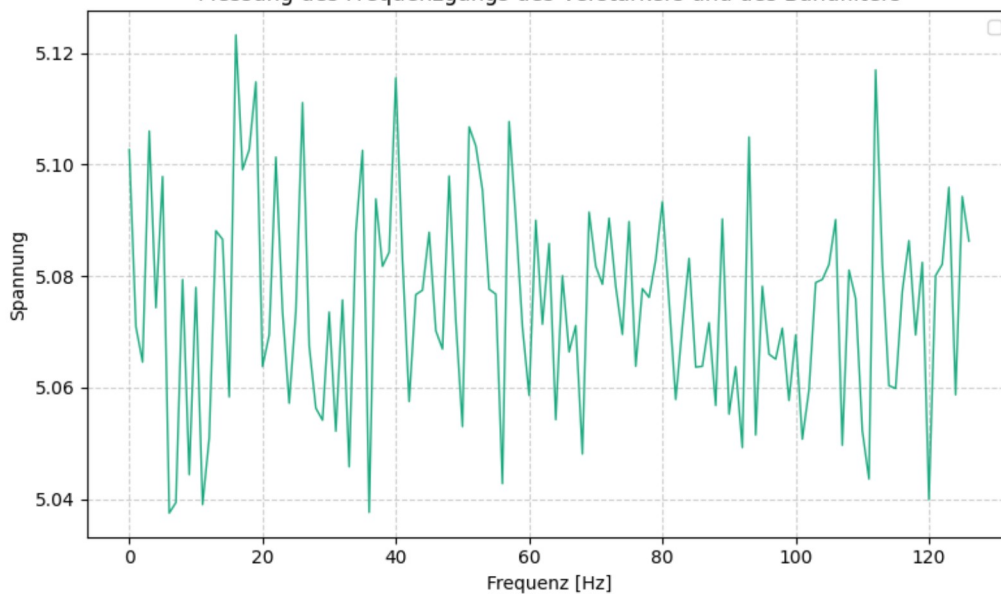


```
In [8]: aufgabe = '2b'
pap.plot_general_csv("A2/30k.csv", "x000", "y000",
                    title=fr"Messung des Frequenzgangs des Verstärkers und des Bandfilters", xlabel="Frequenz [Hz]", ylabel="Spannung",
                    log_x=False, save_path=fr'{plt_save_folder}Frequenzgang_{aufgabe}.pdf')
```

c:\Folders\Uni-Heidelberg\Praktika\PAP2\IDE_PAP2\PAP-2\Versuche\243\Python\Papulator.py:737: UserWarning: No artists with labels found to put in legend.
Note that artists whose label start with an underscore are ignored when legend() is called with no argument.

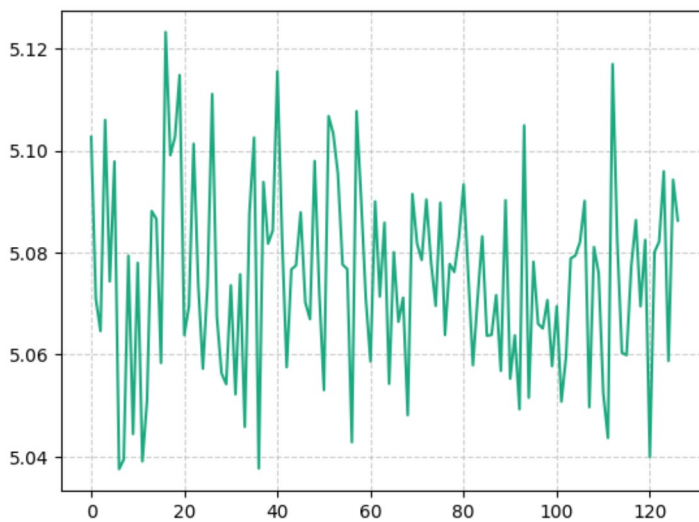
```
ax.legend()
Plot saved to ../img/plots/Frequenzgang_2b.pdf
```


Messung des Frequenzgangs des Verstärkers und des Bandfilters



```
In [9]: df = pd.read_csv("A2/30k.csv", comment='#', engine='python')
x_0 = df['x0000']
y_0 = df['y0000']
plt.plot(x_0, y_0)
```

```
Out[9]: [<matplotlib.lines.Line2D at 0x28a7e53ccb0>]
```



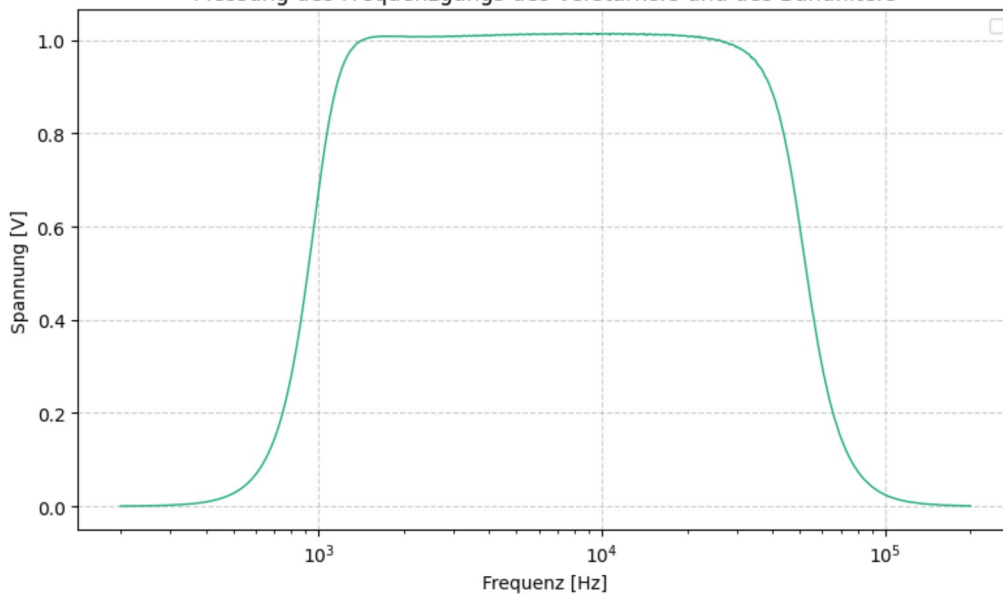
Aufgabe 3

```
In [10]: aufgabe = '3a'
pap.plot_general_csv("A3/Frequenzgang.csv", "Frequency (Hz)", "Channel 1 Magnitude (X)",
                    title=fr"Messung des Frequenzgangs des Verstärkers und des Bandfilters", xlabel="Frequenz [Hz]", ylabel="Spannung [V]",
                    log_x=True, save_path=fr"{plt_save_folder}Frequenzgang_{aufgabe}.pdf")
```

c:\Folders\Uni-Heidelberg\Praktika\PAP2\IDE_PAP2\PAP-2\Versuche\243\Python\Papulator.py:737: UserWarning: No artists with labels found to put in legend. Note that artists whose label start with an underscore are ignored when legend() is called with no argument.

ax.legend()
Plot saved to ../img/plots/Frequenzgang_3a.pdf

Messung des Frequenzgangs des Verstärkers und des Bandfilters



```
In [11]: aufabe = '3c'
df = pd.read_csv("A3/Frequenzgang.csv", comment='#', engine='python')

# print(df)

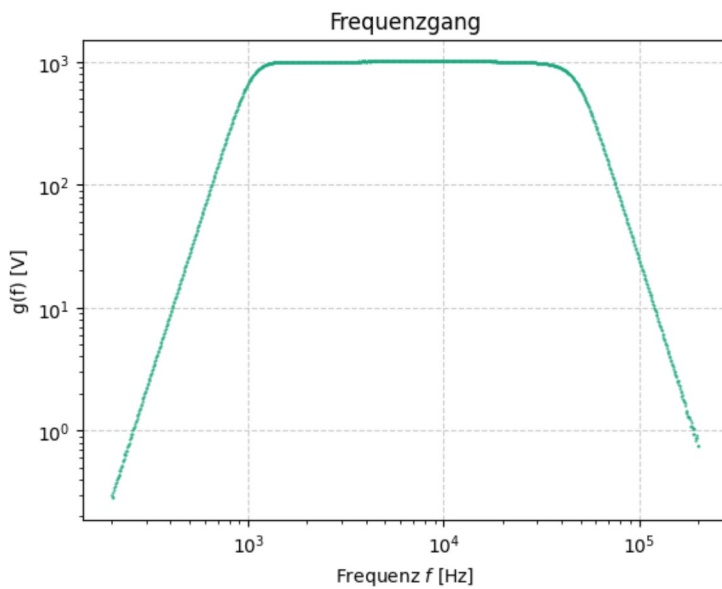
freq_A3 = df['Frequency (Hz)']
gain_rho = df['Channel 1 Magnitude (X)']

damp = 0.001
err_damp = damp * 0.002 # 0.2%

g = gain_rho / damp

plt.loglog(freq_A3,g, linestyle='', marker='.', markersize=1)

# plt.axis([200, 2e5, 10, 1.5E3])
plt.xlabel('Frequenz $f$ [Hz]')
plt.ylabel('g(f) [V]')
plt.title('Frequenzgang')
plt.savefig(fr"{plt_save_folder}Frequenzgang_{aufgabe}.pdf")
plt.show()
```



```
In [12]: def fit_func(f,V,W1,W2,n1,n2):
return V/(np.sqrt(1+1/(f/W1)**(2*n1))*np.sqrt(1+(f/W2)**(2*n2)))
```

```
In [13]: # Fitparameter
# Verstärkung:
ver = 1000

# Grenzfrequenzen
W1 = 1000 #Hz
W2 = 50000 #Hz

# Filterordnungen
n1, n2 = 5, 5

p0 = [ver, W1, W2, n1, n2]

popt, pcov = curve_fit(
    fit_func,
    freq_A3,
    g,
    p0)
```

```
print(fr'Die Verstärkung beträgt: {(pap.round_sig_digs(np.sqrt(pcov[0,0]), popt[0])[2])}')

```

```
U_ver = popt[0] * 1e-3
err_U_ver = np.sqrt(pcov[0,0]) * 1e-1

```

Die Verstärkung beträgt: 1009.4 \pm 0.3

```
In [14]: aufgabe = 'Regression'

f = freq_A3
plt.loglog(
    f,
    fit_func(f, *popt),
    label='Regression'
)

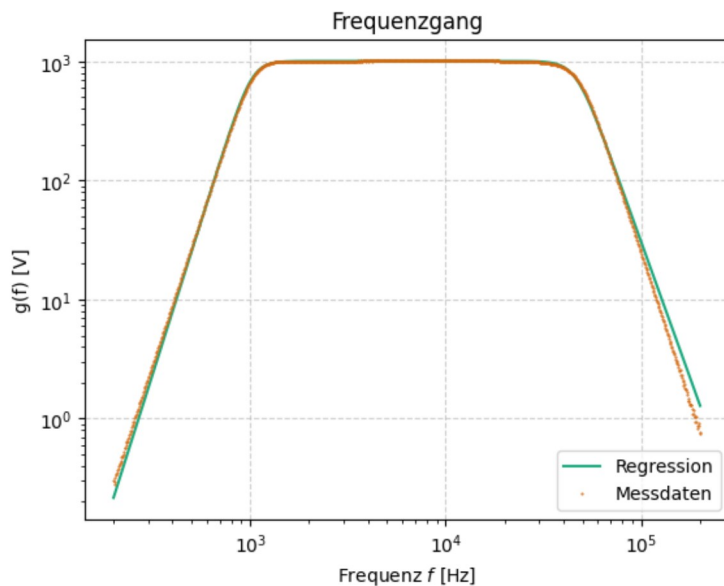
plt.loglog(
    f,
    g,
    linestyle='None',
    marker='.',
    label='Messdaten',
    markersize=1
)

# plt.axis([200, 2e5, 10, 1.5E3])

plt.xlabel('Frequenz $f$ [Hz]')
plt.ylabel('g(f) [V]')
plt.title('Frequenzgang')
plt.legend(loc='best')

plt.savefig(fr"{plt_save_folder}Frequenzgang_{aufgabe}.pdf")

```



```
In [15]: def fit_func_square(f,V,W1,W2,n1,n2):
          return fit_func(f,V,W1,W2,n1,n2)**2

```

```
In [16]: f = np.array(f)
B = quad(fit_func_square, f[0], f[len(f) - 1], args=tuple(popt))
print(fr'Das Integral betraegt: {(pap.round_sig_digs(B[1], B[0])[2])}')
print(fr'{B[0]:.2g}')
print(fr'Der Prozentuale Fehler betraegt 2%, somit: {B[0]*0.02}')
print(fr'Das Integral betraegt: {(pap.round_sig_digs(B[0]*0.02 * 1e-10, B[0] * 1e-10)[2])} * 10^10')

```

Das Integral betraegt: 47209654580 \pm 40

4.7e+10

Der Prozentuale Fehler betraegt 2%, somit: 944193091.6466755

Das Integral betraegt: (4.72 \pm 0.09) * 10^10

Bestimmung der Boltzmannkonstanten

```
In [17]: sym_U_aus, sym_U_V = sp.symbols(r'U_{\text{aus}}, U_{\text{V}}')
diff_func = sym_U_aus**2 - sym_U_V**2

values = np.column_stack([
    avg_Vr[1:], err_Vr[1:],
    [U_ver] * len(avg_Vr[1:]), [err_U_ver] * len(avg_Vr[1:])
])

res = pap.do_it(diff_func, [sym_U_aus, sym_U_V], values, [], False)

U_diff = res[:,0]
err_U_diff = res[:,1]

for i in range(6):
    print(pap.round_sig_digs(err_U_diff[i], U_diff[i])[2])

```

```
In [22]: values = np.array([
            steig * 1e-3, err_steig * 1e-3,
```

```

B[0] / 100,
22.3 + 273.15, 1,
ver, ver * 0.02,
const.R
])

k_b_stat = pap.do_it(func_k_b, [sym.a, sym.B, sym.T, sym.D, sym.R], values, [sym.R, sym.B], False)

print(fr'Statistischer Fehler:{k_b_stat[0][1]:.2g}')

values = np.array([
    steig * 1e-3,
    B[0] , B[0]* 0.02,
    22.3 + 273.15,
    ver,
    const.R
])

k_b_sys = pap.do_it(func_k_b, [sym.a, sym.B, sym.T, sym.D, sym.R], values, [sym.a, sym.T, sym.D, sym.R], False)
print(fr'Systemischer Fehler:{k_b_sys[0][1]:.2g}')

print(fr'Gesamt Fehler: {np.sqrt(k_b_sys[0][1]**2 + k_b_stat[0][1]**2):.2g}')
```

Statistischer Fehler:7.2e-27
Systemischer Fehler:3.6e-29
Gesamt Fehler: 7.2e-27
Systemischer Fehler:3.6e-29
Gesamt Fehler: 7.2e-27

```

In [23]: print(pap.std_abw(val_k_b, const.k_B, err_k_b))
print(const.k_B)

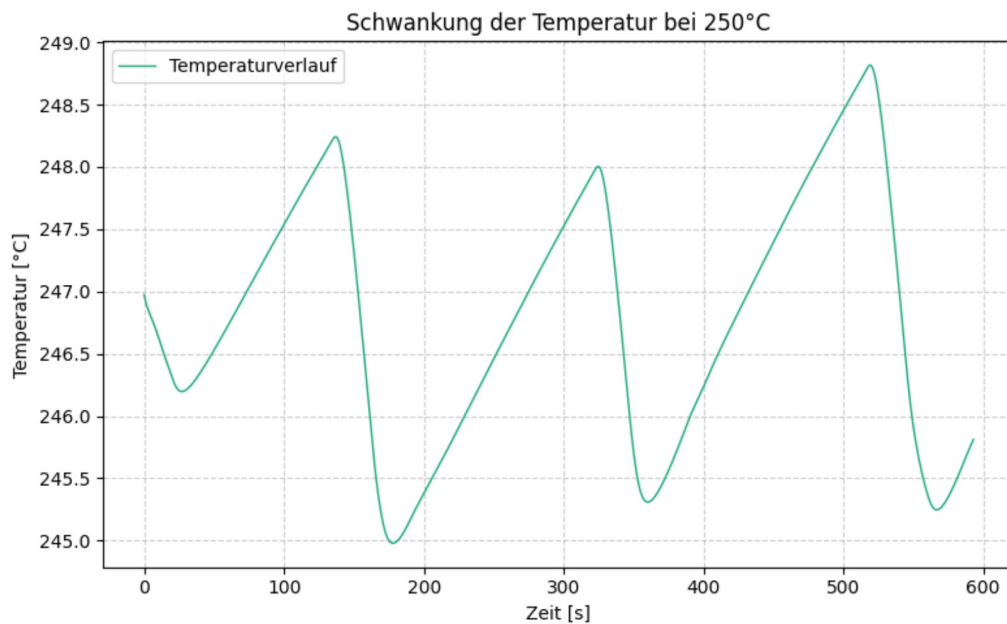
149.22
1.380649e-23
```

Aufgabe 4

```

In [24]: aufgabe = '4a'
pap.plot_general_csv("A4/Schwankung_der_Temperatur_250C.csv", "x000", "y000",
    title=fr"Schwankung der Temperatur bei 250°C", xlabel="Zeit [s]", ylabel="Temperatur [°C]",
    log_x=False, save_path=fr"{plt_save_folder}Frequenzgang_{aufgabe}.pdf",label='Temperaturverlauf')
```

Plot saved to ../img/plots/Frequenzgang_4a.pdf



```

In [25]: # Import der Messdaten
U_diff = U_diff # Bereits bestimmt

temp_theo = np.array([50, 100, 150, 200, 250]) # Grad Celsius (nur zur Info)
avg_Ur_t = np.array([2.4511, 2.7238, 3.0058, 3.297, 3.6351]) * 1e-3 # mV -> V
sig_Ur_t = np.array([0.00922, 0.0113, 0.0115, 0.0132, 0.0135]) * 1e-3 # Fehler mV -> V

# Temperatur in Kelvin
T_K = np.array([52.6, 97.7, 144.8, 188.4, 247.3]) + 273.15

# Widerstand in Ohm (kOhm -> Ohm)
R = np.array([4.8288, 5.5496, 6.3121, 7.027, 8.033]) * 1e3

# Konstanten
B = 4.72 * 1e10 # Hz
err_B = 0.09 * 1e10
```

$$\frac{(U_{aus}^2 - U_V^2)}{R} = c'T$$

$$y = \frac{(U_{aus}^2 - U_V^2)}{R} = (4k_B B)T$$

$$\Rightarrow y/(4 \cdot B) = k_B$$


```

In [26]: def linear_2(a,x,b):
          return a*x+b

In [27]: aufgabe = '4b'
          T_space = np.linspace(-5, 550, 100)
          err_T = 4 #K
          y_data = U_diff[1:] / R

          y_data = avg_Ur_t / R
          err_y = sig_Ur_t / R
          x_data = T_K

          popt_origin, pcov_origin = curve_fit(linear, x_data, y_data, sigma=err_U_diff[1:] * 1e-3, absolute_sigma=True)
          steig_origin = popt_origin[0]
          err_steig_origin = np.sqrt(pcov_origin[0, 0])

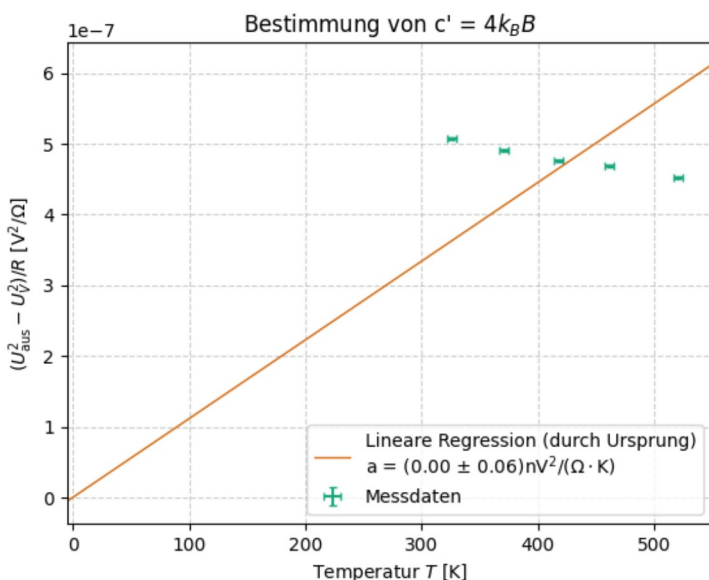
          plt.errorbar(
              x_data,
              y_data,
              err_y,
              err_T,
              label='Messdaten',
              fmt='.',
              ls='',
              capsize=2.5,
              ms = 0.2
          )

          plt.plot(
              T_space,
              linear(T_space, *popt_origin),
              label=f'Lineare Regression (durch Ursprung)\na = ({pap.round_sig_digs(err_steig_origin * 1e6, steig_origin* 1e6)[3]})' r'nV$^2$/($\Omega$ \cdot $K$)',
              linewidth=1
          )

          plt.xlim(-5, 550)

          pap.plot_me('Bestimmung von c\' = $4k_{BB}$', 'Temperatur $T$ [K]', r'$U_{\text{aus}}^2 - U_V^2) / R$ [V$^2$/($\Omega$ K)]', fr"{plt_save_folder}Fit_durch_Urspru

```



```

Out[27]: <Axes: title={'center': "Bestimmung von c' = $4k_{BB}$"}, xlabel='Temperatur $T$ [K]', ylabel='$U_{\text{aus}}^2 - U_V^2) / R$ [V$^2$/($\Omega$ K)]'>

```

```

In [28]: # BEstimmung der Boltzmannkonstante 2
          k_B_zS = steig_origin / 4 / B

          print(f'k_B = {k_B_zS:.2e}')

          k_B = 5.90e-21

```

```

In [29]: aufgabe = '4a'

          y_data = U_diff[1:] / R
          x_data = T_K

          popt_origin, pcov_origin = curve_fit(linear, x_data, y_data, sigma=err_U_diff[1:], absolute_sigma=True)
          steig_origin = popt_origin[0]
          err_steig_origin = np.sqrt(pcov_origin[0, 0])

          # Plot the data and the fit
          plt.figure(figsize=(8, 5))
          x_data_1 = np.linspace(-5, 550, 1000)
          plt.plot(
              x_data_1,
              linear(x_data_1, *popt_origin),
              label=f'Lineare Regression (durch Ursprung)\nc\' = ({pap.round_sig_digs(err_steig_origin * 1e3, steig_origin* 1e3)[2]})' r'mV$^2$/($\Omega$ \cdot $K$)',
              linewidth=2
          )

          popt, pcov = curve_fit(linear_2, x_data, y_data, sigma=err_U_diff[1:], absolute_sigma=True)
          steig = popt[0]
          err_steig = np.sqrt(pcov[0, 0])

          plt.plot(

```

```

x_data_1,
linear_2(x_data_1, *popt),
label=f'Lineare Regression mit angepassten Abschnitt\nc' = ({pap.round_sig_digs(err_steig * 1e3, steig * 1e3)[2]})' r'mV$^2/($\Omega$ \cdot K)',
linewidth=2
)

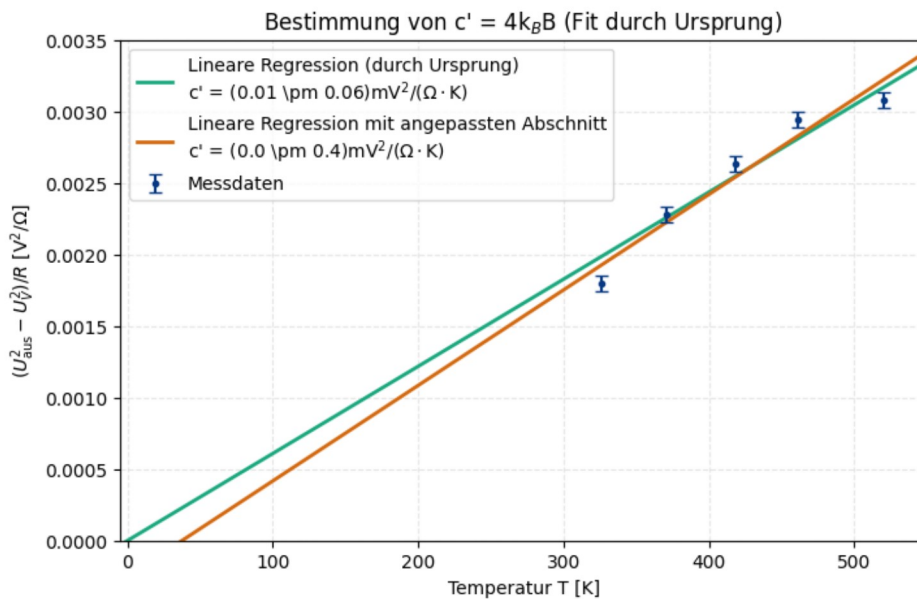
plt.errorbar(
    x_data,
    y_data,
    err_U_diff[1:] * 1e-3,
    label='Messdaten',
    fmt='.',
    ls='',
    capsize=3.5
)

plt.xlabel('Temperatur T [K]')
plt.xlim(-5, 550)
plt.ylim(0, 0.0035)

plt.ylabel(r'$U_{\text{aus}}^2 - U_V^2 / R$ [V$^2/\Omega$]')
plt.title('Bestimmung von c' = 4k_B$B$ (Fit durch Ursprung)')
plt.legend(loc='best')
plt.grid(True, alpha=0.3)
plt.savefig(fr"{plt_save_folder}Fit_durch_Ursprung_{aufgabe}.pdf")
plt.show()

print(f'Steigung (durch Ursprung): {pap.round_sig_digs(err_steig_origin, steig_origin)[2]}')
print(f'c' = 4k_B = {steig_origin:.3g} V^2/(K * Ohm)')

```



Steigung (durch Ursprung): 0.00001 ± 0.00006
 $c' = 4k_B = 6.09 \times 10^{-6} \text{ V}^2 / (\text{K} \cdot \Omega)$

```

In [30]: U_val = avg_Ur_t

y_val = (U_val**2) / R
err_y = (2 * U_val / R) * sig_Ur_t

# Regressionen
def linear_2(x, a, b):
    return a * x + b

def linear(x, a):
    return a * x

# Reg duch (0,0)
params_A, cov_A = optimize.curve_fit(linear, T_K, y_val, p0=[1e-12], sigma=err_y, absolute_sigma=True)
slope_A = params_A[0]
err_slope_A = np.sqrt(cov_A[0][0])

# Freie Parameter (Steigung und Achsenabschnitt)
params_B, cov_B = optimize.curve_fit(linear_2, T_K, y_val, sigma=err_y, absolute_sigma=True)
slope_B, intercept_B = params_B
err_slope_B = np.sqrt(cov_B[0][0])
err_intercept_B = np.sqrt(cov_B[1][1])

# Berechnung von T0 aus Fit B
T0_bestimmt = -intercept_B / slope_B
err_T0 = np.sqrt((err_intercept_B/slope_B)**2 + (intercept_B * err_slope_B / slope_B**2)**2)

# Berechnung von kB aus Fit A
kB_bestimmt = slope_A / (4 * B)
err_kB = err_slope_A / (4 * B)

# Literaturwert für Vergleich
kB_lit = const.k_B

```

```

In [54]: aufgabe = '4c'
# Plot
plt.figure(figsize=(10, 6))
# Plot Daten mit Fehlerbalken
plt.errorbar(
    T_K,

```

```

y_val,
err_y,
4,
fmt='.',
label='Messdaten',
capsize=2.5,
markersize=2,
color = c.BLUE
)

# Fit A Linie
T_fit = np.linspace(-270, 550, 100)

plt.plot(
    T_fit, linear(T_fit, slope_A),
    label=f'Regression durch Ursprung\n$a=({pap.round_sig_digs(err_slope_A * 1e12, slope_A* 1e12)[3]}) \cdot 10^{{-12}}$',
    lw = 1
)

# Fit B Linie
plt.plot(
    T_fit,
    linear_2(T_fit, slope_B, intercept_B),
    label=f'Regression mit freien Parametern\n$a=({pap.round_sig_digs(err_slope_B * 1e12, slope_B* 1e12)[3]}) \cdot 10^{{-12}}$',
    lw = 1
)

plt.xlim(-270, 550)
plt.ylim(-0.1e-9, 2e-9)

plt.hlines(0,
    -270,
    550,
    color=c.WINE_RED,
    label='Null-Linie',
    lw = 1
)

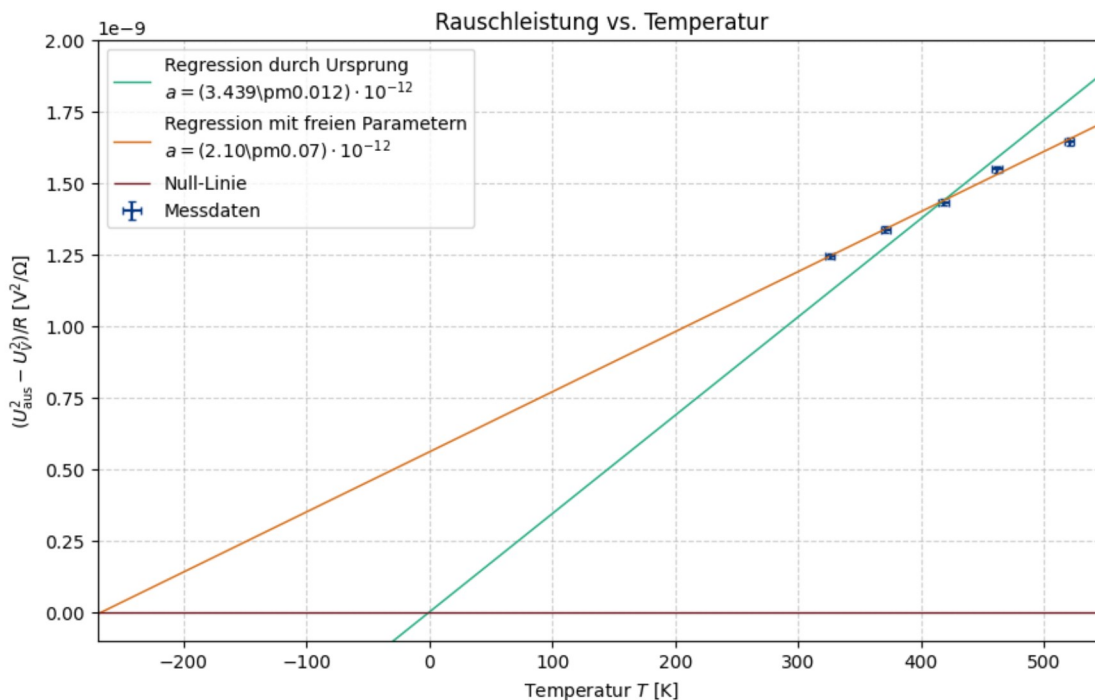
pap.plot_me('Rauschleistung vs. Temperatur', 'Temperatur $T$ [K]', r'$U_{\text{aus}}^2 - U_V^2 / R$ [$V^2\$/\Omega$]', fr'{plt_save_folder}Fit_durch_Un:

```

```

<>:22: SyntaxWarning: invalid escape sequence '\c'
<>:30: SyntaxWarning: invalid escape sequence '\c'
<>:22: SyntaxWarning: invalid escape sequence '\c'
<>:30: SyntaxWarning: invalid escape sequence '\c'
C:\Users\finnz\AppData\Local\Temp\ipykernel_6984\3172382637.py:22: SyntaxWarning: invalid escape sequence '\c'
    label=f'Regression durch Ursprung\n$a=({pap.round_sig_digs(err_slope_A * 1e12, slope_A* 1e12)[3]}) \cdot 10^{{-12}}$',
C:\Users\finnz\AppData\Local\Temp\ipykernel_6984\3172382637.py:30: SyntaxWarning: invalid escape sequence '\c'
    label=f'Regression mit freien Parametern\n$a=({pap.round_sig_digs(err_slope_B * 1e12, slope_B* 1e12)[3]}) \cdot 10^{{-12}}$',

```



```

Out[54]: <Axes: title={'center': 'Rauschleistung vs. Temperatur'}, xlabel='Temperatur $T$ [K]', ylabel='$U_{\text{aus}}^2 - U_V^2 / R$ [$V^2\$/\Omega$]'>

```

```

In [78]: print("Ergebnisse\n")

print(fr"Fit A (durch Null): Steigung a = {slope_A:.3e} \pm {err_slope_A:.3e}")

print(fr"Berechnete kB (Fit A): {pap.round_sig_digs(err_kB * 1e23, kB_bestimmt * 1e23)[2]} J/K")

print(fr"Literaturwert kB: {kB_lit} J/K")
print(fr"Abweichung (%) : {abs(kB_bestimmt - kB_lit)/kB_lit * 100:.1f}%")
print(fr"Die Standardabweichung betrgt: {pap.std_abw(const.k_B,kB_bestimmt,err_kB)}")

print('\n', "-" * 20, '\n')

print(fr"Fit B (frei): Steigung a = {slope_B:.3e}, Achsenabschnitt b = {intercept_B:.3e}")

print(fr"Bestimmter absoluter Nullpunkt T0: {pap.round_sig_digs(err_T0, T0_bestimmt)[2]} K")

```

```
print(fr"Literaturwert T0: 0 K")
print(fr"Differenz zu 0 K: {T0_bestimmt:.1f} K")
print(fr"Die Standardabweichung betragt: {pap.std_abw(0,T0_bestimmt,err_T0)}")
```

Ergebnisse

Fit A (durch Null): Steigung a = 3.439e-12 \pm 1.196e-14
Berechnete kB (Fit A): 1.821 \pm 0.006 J/K
Literaturwert kB: 1.380649e-23 J/K
Abweichung (%) : 31.9%
Die Standardabweichung betragt: 69.57

Fit B (frei): Steigung a = 2.101e-12, Achsenabschnitt b = 5.597e-10
Bestimmter absoluter Nullpunkt T0: -266 \pm 17 K
Literaturwert T0: 0 K
Differenz zu 0 K: -266.4 K
Die Standardabweichung betragt: 15.82

In [79]: `pap.std_abw(1.44, 1.821, 0.3, 0.1)`

Out[79]: `np.float64(1.2)`